

Y5 Promo Exam Paper 2 Marksheet

		Mark		Remarks
Task 1.1 load_players()	File handling with open/read/close	1	0	show that file is closed
	Loops through each line in the file	1	1	
	Processes each line into fields (split,strip)	1	1	
	Store each player's data in correct order and returns the list	1	1	
	Displays first 10 players from the loaded list	1	1	
	Sub-total	5	4	
Task 1.2 quicksort()	Defines quicksort() with suitable parameters (arr, low, high)	1	1	
	Implements partition() function correctly, translating pseudocode	1	1	
	Sorts by score in descending order	1	0	compare the score part only, not the sublist
	Handles tie-breaker by username in ascending alphabetical order	1	0	
	Makes recursive calls correctly on sublists (low..index-1 and index+1..high)	1	1	
	Displays first 10 players before and after sorting to demonstrate correctness	1	0	incorrect output
	Sub-total	6	3	
Task 1.3 get_rank()	Defines function get_rank() with suitable parameters and returns an integer	1	1	
	Correctly searches for the username in the sorted leaderboard	1	1	
	Correctly handles ties (players with the same score share the same rank)	1	0	
	Correctly handles skipped ranks for lower scores after ties	1	0	
	Demonstrates testing by finding and displaying the rank of "PixelWizard"	1	0	
	Sub-total	5	2	
Task 1.4 find_players_in_range()	Defines function with three given parameters (leaderboard, min_score, max_score)	1		
	Uses binary search to locate the position of min_score (or nearest larger score)	1		
	Uses binary search to locate the position of max_score (or nearest smaller score)	1		
	Binary search logic implemented for descending order elements...	1		
	...(step updates, loop termination, comparisons, correct index calculation)	1		
	Builds sublist of players between the located start and end indices (inclusive)	1		
	Returns a list of player records (not just indices or scores)	1		
	Displaying usernames,scores,emails of players with scores between 1100 and 1200	1		
	Sub-total	8	0	no evidence
Task 2.1	Defines the Node class with the attributes SongName and Pointer	1	1	
	Implements getter methods for both SongName and Pointer	1	1	
	Implements setter methods for both SongName and Pointer	1	1	
	Sub-total	3	3	
Task 2.2 InsertSong() RemoveSong() Reverse() Search() Display()	Correctly defines Head, Tail, and Length in __init__	1	1	
	Validates position is between 1 and Length+1; prints/returns "Invalid position" if not	1	1	Length cannot be calculated using Head and Tail
	Inserts song at front (position 1) correctly	1	0	
	Inserts song at middle positions correctly, updating pointers	1	0	
	Inserts song at end (position Length+1) correctly	1	0	
	Correctly updates Length after insertion	1	0	
	Maintains proper pointers (Head/Tail) for all nodes after insertion	1	0	
	Searches and finds the first occurrence of the song	1	0	
	Removes the node correctly (updating pointers for Head, Tail, or middle nodes)	1	0	
	Updates Length after removal	1	0	
	Prints "<songname> not found" if the song is not in the playlist or handles edge cases	1	0	
	Reverses the playlist by changing pointers correctly	1	0	
	Updates Head and Tail after reversing	1	0	
	Works for playlists of any length, including empty or single-node playlists	1	0	
	Returns the position of the first occurrence of the song (case-sensitive)	1	0	
	Returns -1 if the song is not found	1	0	
	Checks if the playlist is empty and prints "Empty Playlist" if so	1	0	
	Traverses the linked list from Head to Tail correctly	1	0	
	Output all song names in order	1	0	
	Prints the song names separated by " -> "	1	0	
	Sub-total	20	2	
Task 2.3	Performs insertions correctly	1	0	
	Performs removal correctly	1	0	
	Displays the playlist correctly after all operations	1	0	
	Sub-total	3	0	
Task 3.1 BST class insert()	Correctly includes the TreeNode class	1		
	Defines BinarySearchTree class and initialises root to None	1		
	Correctly handles node is None -> returns TreeNode(key)	1		
	Correctly uses setLeft/setRight based on comparison of key with node's key	1		
	Recursively inserts into the left subtree when key < node.getKey()	1		
	Recursively inserts into the right subtree when key > node.getKey()	1		
	Detects duplicate key and prints "Duplicate key <key> ignored"	1		
	Returns the current node after insertion	1		
	Sub-total	8	0	no evidence
Task 3.2 in_order_traversal()	Defines the in_order_traversal method inside BinarySearchTree.	1		
	Correctly handles the base case (if node is None) to stop recursion.	1		
	Traverses the left subtree before visiting the current node.	1		
	Appends the current node's key to the result list.	1		
	Traverses the right subtree and returns the result list at the end.	1		
	Sub-total	5	0	no evidence
Task 3.3	Creates a BST object and inserts keys in order	1		
	Attempts to insert duplicate key 22 and trigger duplicate handling message	1		
	Performs an in-order traversal using in_order_traversal and prints the resulting list	1		
	Sub-total	3	0	no evidence
Good programming style	Meaningful identifiers used	1	1	
	Use of appropriate whitespace and formatting	1	1	
	At least 3 comments were used to explain code logic for each Task	1	0	
	Comments used to explain code logic in more than 50% code	1	0	
	Sub-total	4	2	
TOTAL		70	16	